

الـ **REST API** (أو Representational State Transfer) هي ببساطة "مترجم ودليفري" في نفس الوقت. هي الطريقة القياسية التي يتخلى أي برنامجيّن أو نظامين مختلفين يتكلموا مع بعض من غير ما يعرفوا تفاصيل بعض.

يعني الـ Frontend (الـ UI التي المستخدم بيشفه) يقدر يتكلم مع الـ Backend (السيرفر والداتا بيز) من خلالها، حتى لو ده مكتوب بلغة وده بلغة ثانية خالص.

يعني إيه REST API بالبلدي؟ (مثال المطعم)

عشان تفهمها وماتنسهاش، تخيل إنك قاعد في مطعم:

1. الـ **Client** (الزبون): هو أنت (الـ Browser أو الموبايل أبليكيشن).

2. الـ **Server** (المطبخ): فيه الأكل والداتا كلها (الـ Database).

3. الـ **API** (الويتر/الجرسون): بياخد طلبك من الترابيزة، يروح بيه للمطبخ، ويجيب لك الأكل لحد عندك.

الـ REST بقى هي "قواعد الاتكيت" بتاعة الويتر ده. يعني نظام متفق عليه عالمياً عشان الطلبات والردود تمشي بنظام من غير عك.

ليه بنستخدم الـ REST APIs؟ (المميزات والفوائد)

من غير الـ APIs، كانت برمجيات الويب هتبقى جزر معزولة. إحنا بنستخدمها للأسباب الجوهرية دي:

1. فصل القوات (Separation of Concerns)

الـ Frontend ملوش دعوة الداتا متسيفة فين ولا الـ Backend مكتوب بـ Node.js ولا Python. الـ Frontend كل اللي عليه يطلب الداتا ويعرضها، والـ Backend عليه يجيب الداتا ويبعتها. ده بيخلي تعديل أي طرف سهل من غير ما يهد الطرف الثاني.

2. تعدد المنصات (Cross-Platform)

لو عندك داتا بيز للمنتجات، بتعمل API واحدة بس. الـ Web App بتاعك بيكلم نفس الـ API، ومستقبلاً لو عملت Android App أو iOS App هيكلوا نفس الـ API برضه. مش هتضطر تعيد كتابة كود الـ Backend لكل منصة.

3. لغة موحدة (Stateless & JSON)

الـ REST APIs بتنقل البيانات غالباً بصيغة **JSON** (وهي مجرد طريقة كتابة بيانات بسيطة ومفهومة لكل لغات البرمجة). كمان هي **Stateless**، يعني السيرفر مش محتاج يفكر الطلب القديم بتاعك، كل **Request** بيبعت معاه بياناته كاملة والموضوع بيخلص في وقته.

4. دمج الخدمات الخارجية (Third-Party Integrations)

مش كل حاجة هتبنيتها بنفسك! عايز تعرض خريطة في موقعك؟ بتكلم الـ API بتاعة Google Maps. عايز الزبون يدفع بالفيزا؟ بتكلم الـ API بتاعة Stripe أو Fawry. الـ API هي التي بتخليك تركيب خدمات جاهزة في موقعك في ثواني.

أشهر ثلاث صيغ هتقابلهم في حياتك هما: JSON، و XML، و YAML (وفي حاجات تانية حديثة زي Protocol Buffers).

1. الـ JSON (JavaScript Object Notation)

الملك المتوج حالياً في عالم الـ Web والـ REST APIs.

- **فكرته:** مبني على طريقة كتابة الـ Objects في لغة JavaScript (لكنه بيشتغل مع كل اللغات). بيعتمد على الـ `key`: `value`.
- **شكله:**

JSON

```
{
  "id": 101,
  "name": "Ibrahim",
  "skills": ["Frontend", "Fullstack"],
  "isActive": true
}
```

- **المميزات:** خفيف جداً، سريع في القراءة والكتابة (Parsing)، ومفهوم بمجرد النظر.
- **العيوب:** مفيش فيه كومنتات (Comments)، ومش بيدعم أنواع بيانات معقدة أوي زي التواريخ بشكل مباشر (بتتبعث كـ String).

2. الـ XML (eXtensible Markup Language)

الجد القديم اللي لسه مسيطر على الأنظمة الضخمة (Legacy Systems) والـ SOAP APIs، وبعض تكوينات الأندرويد.

- **فكرته:** شبه الـ HTML، بيعتمد على الـ `<tag>...</tag>` Tags (عشان يعرف البيانات).
- **شكله:**

XML

```
<user>
  <id>101</id>
  <name>Ibrahim</name>
  <skills>
    <skill>Frontend</skill>
    <skill>Fullstack</skill>
  </skills>
  <isActive>true</isActive>
</user>
```

- **المميزات:** منظم جداً، بيدعم الـ Validation القوي (عن طريق حاجة اسمها Schema أو DTD)، وبيدعم الكومنتات.

- **العيوب:** "رغاي" والكتابة فيه كثيرة (Verbose)، حجم الملف يبقى كبير بسبب الـ Closing Tags، وبطيء في الـ Parsing مقارنة بـ JSON.

3. الـ YAML (YAML Ain't Markup Language)

صيغة مخصصة أكثر لملفات التكوين (Configuration) زي ملفات الـ Docker، الـ CI/CD pipelines، وبعض الـ APIs (زي مواصفات OpenAPI/Swagger).

- **فكرته:** يعتمد تماماً على المسافات (Indentation) والـ Newlines عشان يحدد العلاقات، من غير أقواس ولا Tags.
- **شكلة:**

YAML

```
id: 101
name: Ibrahim
skills:
  - Frontend
  - Fullstack
isActive: true
```

- **المميزات:** أنصف وأسهل صيغة للقراءة البشرية، ويدعم الكومنتات.
- **العيوب:** المسافة الغلط تبوظ الملف كله (حساس جداً للـ Spaces)، والـ Parsers بتاعته أبطأ وأعقد.

1. الـ Endpoint والـ Routing (العنوان والخريطة)

- **الـ Endpoint:** هو الـ URL المحدد اللي الـ Client بيعت عليه الـ Request عشان يوصل لـ Resource (بيانات معينة).
- **الـ Routing:** هو الآلية أو الكود اللي جوه الـ Backend اللي بيقرأ الـ Endpoint دي، ويوجّه الـ Request للـ Function الصح اللي هتنفذ المطلوب.

تشرح الـ URL (Anatomy of a REST URL)

تخيل عندك الـ Endpoint ده:

`https://api.voot.com/v1/users/101/orders?status=shipped`

- **Base URL:** الـ `https://api.voot.com` (السيرفر بتاعك).
- **Versioning:** الـ `/v1/` (تفضل شغالة على) مستقبلاً تضمن إن التطبيقات القديمة متقلش API مهم جداً عشان لو عدلت الـ (v2) والجديد يروح لـ v1).
- **Resource:** الـ `/users/` (وليس فعل Plural الكيان الأساسي اللي بتتعامل معاه، ولازم يكون اسم جمع).
- **Path Parameter:** الـ `/101/` دا متغير بيعبر عن ID أو معرف فريد لحاجة معينة (هنا المستخدم رقم 101).

- /orders : Sub-resource (الطلبات التابعة للمستخدم 101).
- **Sorting** أو **Filtering** بتبدأ بعد علامة ؟ وبُستخدم للـ **Query Parameters** : `?status=shipped` الـ **Pagination** نفسه Resource مش عشان تحدد الـ ، (الترتيب والتصفية والصفحات)

2. الـ HTTP Methods Deep Dive (الأفعال الإجرائية)

في الـ REST، إحنا مش بنعمل Endpoint لكل حركة (يعني غلط جداً نكتب `delete-user/` أو `update-user/`). الـ Endpoint بيفضل ثابت كاسم: `users/` ، واللي بيحدد الفعل هو الـ **HTTP Method**.

تعال نمسكهم واحدة واحدة بالتفصيل التقني (مفهوم الـ Idempotency والـ Safety):

1 GET (قراءة البيانات)

- **وظيفتها:** بتطلب داتا من السيرفر.
- **الـ Body:** مفيش فيها Request Body (البيانات بتبع في الـ URL نفسه لو محتاج فلتر).
- **Safe:** نعم. (يعني لو ناديتها مليون مرة مش هتغير حاجة في الداتا بيز).
- **Idempotent:** نعم. (هيرجع نفس النتيجة طالما الداتا متعدلتش من بره Request نفس الـ).
- **أمثلة:**
 - `GET /v1/users` (هات كل المستخدمين) -> الرد 200 OK .
 - `GET /v1/users/101` (هات بيانات المستخدم 101).

2 POST (إنشاء بيانات جديدة)

- **وظيفتها:** بتبع داتا في الـ **Body** عشان تكرت الـ Resource لأول مرة.
- **Safe:** لا. (بتغير في حالة الداتا بيز وبتزود ريكورد).
- **Idempotent:** لا. (لو دوست على زرار "تسجيل" مرتين ورا بعض، السيرفر هيكريت مستخدمين اثنين بنفس البيانات، إلا لو عامل حماية ضد التكرار).
- **الرد المثالي:** 201 Created مع الـ الـ Object الجديد اللي اتعمله مع الـ ID بتاعه.
- **مثال:** `POST /v1/users` والـ Body فيه بياناته.

3 PUT (تعديل كامل / إحلال)

- **وظيفتها:** بتعديل الـ Resource بالكامل. يعني بتبع الـ Object الجديد كامل، والسيرفر بيعمل إحلال (Replace) للتقديم بالجديد. لو بعث بيانات ناقصة، الباقي هيتمسح أو هيبقى `null`.
- **Safe:** لا.
- **Idempotent:** نعم. (لو بعث نفس التعديل الكامل 10 مرات، النتيجة النهائية على السيرفر هي من أول مرة).
- **مثال:** `PUT /v1/users/101` (لازم تبع كل الحقول حتى لو هتعدل الإسم بس).

4 PATCH (تعديل جزئي)

- **وظيفتها:** تعديل حقول معينة فقط (Partial Update). بتبع بس الحقل اللي عايز تغيره والسيرفر بيعدله ويسيب الباقي زي ما هو.

- **Safe:** لا.
- **Idempotent:** Counter زي لو بتتبع أمر يزود الـ) في بعض الحالات Idempotent غالباً نعم، بس تقنياً ممكن متكونش (هيزود المجموع Request بمقدار 1، فكل مثال: PATCH /v1/users/101 والـ Body فيه {"status": "premium"} بس.

5 DELETE (حذف البيانات)

- **وظيفتها:** مسح الـ Resource المحدد.
- **Safe:** لا.
- **Idempotent:** نعم. (OK 204 أو No Content . 404 . Not Found المرات اللي بعدها هيرد 404 . نعم 200) . نعم (Found Resource لأن الـ Found (اتمسح خلاص، لكن الحالة على السيرفر مش هتتغير؛ هو ممسوح بالفعل Resource لأن الـ Found . مثال: DELETE /v1/users/101 .

الـ REST مش لغة برمجة ولا Framework، الـ REST هي فلسفة تصميم (Architectural Style). طول ما أنت محقق الـ 5 مبادئ الإجبارية دول، أنت كده بتعمل REST API على أبوه!

1. فصل العميل عن الخادم (Client-Server Architecture)

- **الفكرة:** الفصل التام بين الـ UI (الـ Client) والـ Data Storage (الـ Server).
- **التطبيق:** الـ Client ملوش دعوة الداتا بيز نوعها إيه ولا السيرفر شغال إزاي، هو ببيعت Request وبس. والـ Server ملوش دعوة بشكل الشاشة أو الـ UI عامل إزاي.
- **الميزة:** تقدر تغير كود الـ Frontend كله أو تقلب الأبليكيشن Mobile App من غير ما تلمس خط كود واحد في الـ Backend.

2. عدم حفظ الحالة (Statelessness)

- **الفكرة:** السيرفر ألزهايمر! مش بيفتكر أي حاجة من الـ Request القديم.
- **التطبيق:** كل Request ببيعت الـ Client لازم يكون "مكتفي ذاتياً" (Self-contained). يعني شايل معاه كل البيانات المطلوبة للتنفيذ (زي الـ Token بتاع الـ Authentication، والـ ID، وكل حاجة). السيرفر مش بيسيف الـ Session عندك.
- **الميزة:** السيرفر بيبقى خفيف جداً، وتقدر تكبر السيرفرات وتعمل (Load Balancing) براحتك لأن أي سيرفر يقدر يرد على أي Request في أي وقت.

3. القابلية للتخزين المؤقت (Cacheability)

- **الفكرة:** مش كل ما الـ Client يعوز داتا يروح يخط على السيرفر ويجهد الداتا بيز.
- **التطبيق:** الـ Response اللي جاي من السيرفر لازم يحدد صراحة (جوه الـ HTTP Headers) هل الداتا دي ينفع الـ Client يعملها Cache (تخزين مؤقت) ولفترة قد إيه، ولا لأ.
- **الميزة:** بتوفر Bandwidth وتقلل الضغط على السيرفر، والموقع بيبقى طلاقة عند المستخدم (زي صور المنتجات أو قوائم البلاد اللي مابتتغيرش كتير).

4. الواجهة الموحدة (Uniform Interface)

- **الفكرة:** ده أهم مبدأ وميز مبرمجي الـ REST. لازم شكل الـ API وطريقة طلب الداتا تكون موحدة ومفهومة لأي حد في العالم بمجرد النظر.
- **التطبيق:** وبيتحقق بـ 4 حاجات:
 - **Identification of resources:** (زي `/products` / زي) الجمع للتعبير عن الكيانات URLs استخدام الـ
 - **Manipulation of resources through representations:** الـ Client لما بيعت تمثيل الـ Client الـ (مع ID مع JSON body زي) للداتا
 - **Self-descriptive messages:** الـ Request والـ Response بيشرحوا نفسهم (زي تحديد) `Content-Type: application/json`.
 - **HATEOAS:** الـ (الـ) Response اختصار مجلص معناه إن الـ (الـ) (تقدر تعملها).

5. نظام الطبقات (Layered System)

- **الفكرة:** الـ Client أعمر، مش عارف هو بيكلم السيرفر الحقيقي ولا وسيط.
- **التطبيق:** الـ API بتسمحك تحط طبقات في النص بين الـ Client والسيرفر الأساسي (زي طبقة أمان Firewall، أو Proxy، أو Load Balancer، أو سيرفر للـ Caching).
- **الميزة:** بتزود الأمان والـ Scalability للسيستم من غير ما الـ Client يحس بأي تغيير في طريقة طلب الداتا.

6. الكود عند الطلب (Code on Demand) - اختياري

- **الفكرة:** السيرفر يقدر بيعت كود حقيقي يتنفذ عند الـ Client.
- **التطبيق:** بدل ما السيرفر بيعت JSON بس، ممكن بيعت سكريبت JavaScript أو Applet يتنفذ جوه البراوزر بتاع الـ Client.
- **الميزة:** ده المبدأ الوحيد الاختياري في الـ REST، ونادراً جداً لما حد بيستخدمه حالياً لأن الـ Frontends بقت ذكية وبتشيل منطق التشغيل بنفسها.

إيه هو الـ CORS وإيه سبب الـ Error؟

(مشاركة الموارد بين الأصول المختلفة) Cross-Origin Resource Sharing اختصار لـ CORS

دي مش غباوة من المتصفح، دي ميزة أمان عالية جداً جوه الـ Browser (شغالة بمبدأ اسمه Same-Origin Policy). المتصفح بطبعه "شكاك"، لو موقعك شغال على عنوان (Origin) زي `http://localhost:3000` (الـ React)، ورايح تطلب داتا من سيرفر شغال على عنوان تاني خالص زي `http://localhost:5000` (الـ Node.js)، المتصفح هيقف الـ Request فوراً!

المتصفح بيروح يسأل السيرفر الأول: "يا سيرفر، الـ Frontend الغريب ده ينفع ياخد منك داتا؟" لو السيرفر ماردش عليه وقاله "آه أنا عارفه وموافق"، المتصفح بيقل السكة ويرمي لك الـ CORS Error في الـ Console.

يعني حل المشكلة دي دائماً بيكون من ناحية السيرفر (Backend)، مش ال Frontend. السيرفر لازم بيعت موافقة للمتصفح في ال Headers.

عشان السيستم يشتغل صح، لازم كل حاجة تروح في ال Header الفريد بتاعها، كالتالي:

JavaScript

```
app.use((req, res, next) => {
  // 1. (بتاعك React هنا بنقول كله، أو حط رابط ال) مين مسموح له يدخل؟
  res.setHeader("Access-Control-Allow-Origin", "*");

  // 2. المسموح بيها؟ (HTTP Methods) إيه الأفعال
  res.setHeader("Access-Control-Allow-Methods", "GET, POST, PUT, PATCH, DELETE, OPTIONS");

  // 3. Request مسموح له يبعثها في ال Frontend الي ال Headers إيه ال
  res.setHeader("Access-Control-Allow-Headers", "Content-Type, Authorization");

  // رده بـ ، (المتصفح بيتيست بيه السكة) OPTIONS نوعه Request مهم جداً: لو ال
  // 200 فوراً وماتكملش
  if (req.method === 'OPTIONS') {
    return res.sendStatus(200);
  }

  next();
});
```

لو مش عوز تكتب كل ده؟ (الحل السحري)

في Node.js، فيه مكتبة جاهزة بتخلص الحوار ده كله بسطرين بدل العك ده كله. بتنزل حزمة اسمها cors :

```
npm install cors
```

وتكتب في كودك سطر واحد بس:

JavaScript

```
const cors = require('cors');
app.use(cors()); // كده فتحتها بحري لكل الناس وبشكل صحيح 100
```